Unit 3

Advanced Statistics

Topics

- Point estimates
- Sampling distributions
- Confidence intervals
- Hypothesis tests
  - ✔ Conducting a hypothesis test
  - ✔ One sample t-tests – Example, Assumptions
  - ✔ Type I and type II errors
  - ✔ Hypothesis test for categorical variables - Chi-square goodness of fit test - Chi-square test for association/independence

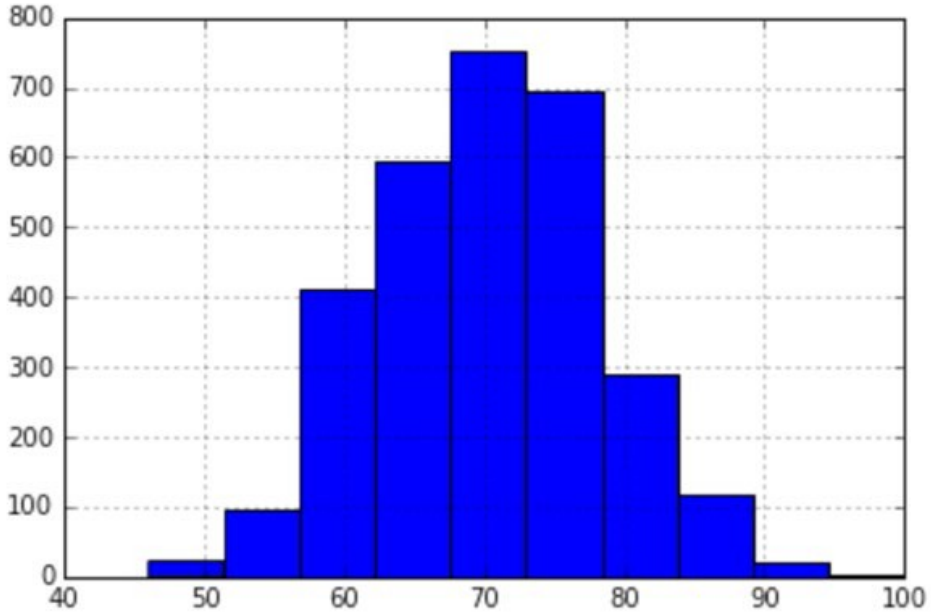
Point Estimates

- It was difficult to obtain a population parameter; so, we had to use sample data to calculate a statistic that was an estimate of a parameter. When we make these estimates, we call them point estimates.
- A **point estimate** is an estimate of a population parameter based on sample data.
- We use point estimates to estimate population means, variances, and other statistics.
- To obtain these estimates, we simply apply the function that we wish to measure for our population to a sample of the data.
- For example, suppose there is a company of 9,000 employees and we are interested in ascertaining the average length of breaks taken by employees in a single day. As we probably cannot ask every single person, we will take a sample of the 9,000 people and take a mean of the sample. This sample mean will be our point estimate.

- The following code is broken into three parts:
  - We will use the probability distribution, known as the Poisson distribution, to randomly generate 9,000 answers to the question: for how many minutes in a day do you usually take breaks? This will represent our "population". The **Poisson random variable** is used when we know the average value of an event and wish to model a distribution around it.
  - We will take a sample of 100 employees (using the Python random sample method) and find a point estimate of a mean (called a sample mean).
  - Compare our sample mean (the mean of the sample of 100 employees) to our population mean.

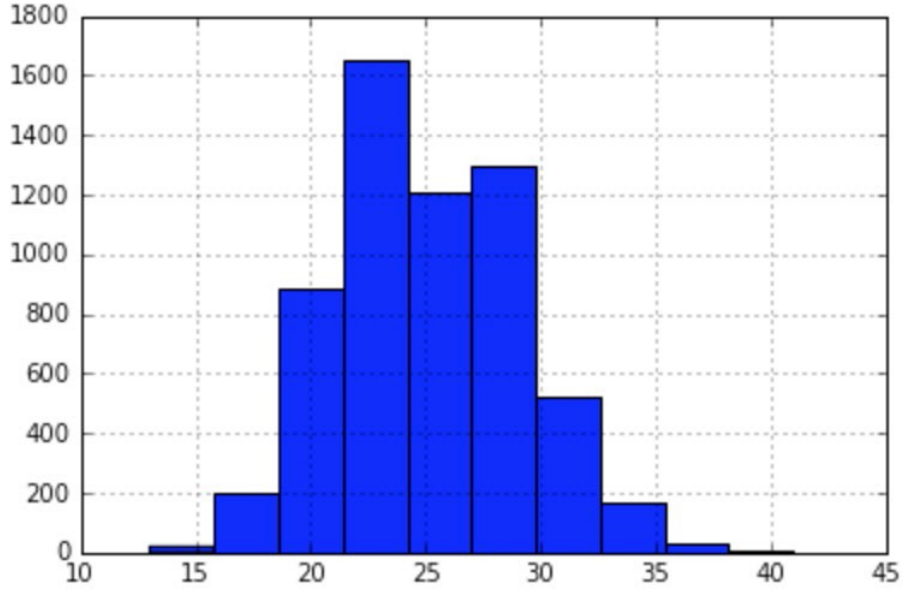
```
np.random.seed(1234)
long_breaks = stats.poisson.rvs(loc=10, mu=60,
size=3000)
# represents 3000 people who take about a 60 minute
break
```

- The long_breaks variable represents 3000 answers to the question: *how many minutes on an average do you take breaks for?*, and these answers will be on the longer side. Let's see a visualization of this distributi
- pd.Series(long_breaks).hist()
- We see that our average of 60 minutes is to the left of the distribution. Also, because we only sampled 3000 people, our bins are at their highest around **700-800** people.



- Now, let's model 6000 people who take, on an average, about 15 minutes' worth of breaks. Let's again use the Poisson distribution to simulate 6000 people, as shown:

```
short_breaks = stats.poisson.rvs(loc=10, mu=15, size=6000)
# represents 6000 people who take about a 15 minute break
pd.Series(short_breaks).hist()
```



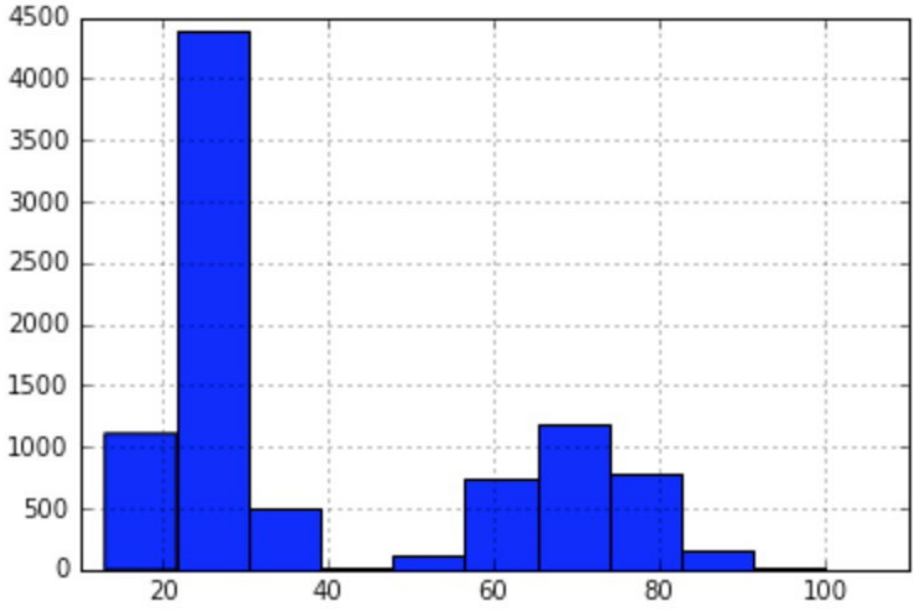
- So, we have a distribution for the people who take longer breaks and a distribution for the people who take shorter breaks. Again, note how our average break length of 15 minutes falls to the left-hand side of the distribution, and note that the tallest bar is about **1600** people

```
breaks = np.concatenate((long_breaks, short_breaks))
# put the two arrays together to get our "population" of 9000 people
```

- The breaks variable is the amalgamation of all the 9000 employees, both long and short break takers. Let's see the entire distribution of people in a single visualization:

```
pd.Series(breaks).hist()
```

- We see how we have two humps. On the left, we have our larger hump of people who take about a 15 minute break, and on the right, we have a smaller hump of people who take longer breaks.



- We can find the total average break length by running the following code:

  ```
  breaks.mean()
  # 39.99 minutes is our parameter.
  ```

- Our average company break length is about 40 minutes. Remember that our population is the entire company's employee size of 9,000 people, and *our parameter is 40 minutes*.
- In the real world, our goal would be to estimate the population parameter because we would not have the resources to ask every single employee in a survey their average break length for many reasons. Instead, we will use a point estimate.

- So, to make our point, we want to simulate a world where we ask 100 random people about the length of their breaks. To do this, let's take a random sample of 100 employees out of the 9,000 employees we simulated, as shown:

```
sample_breaks = np.random.choice(a = breaks, size=100)
# taking a sample of 100 employees
```

- Now, let's take the mean of the sample and subtract it from the population mean and see how far off we were:

```
breaks.mean() - sample_breaks.mean()
# difference between means is 4.09 minutes, not bad!
```

- This is extremely interesting, because with only about 1% of our population (100 out of 9,000), we were able to get within 4 minutes of our population parameter and get a very accurate estimate of our population mean. Not bad!
- Here, we calculated a point estimate for the mean, but we can also do this for proportion parameters. By proportion, I am referring to a ratio of two quantitative values.

- Let's suppose that in a company of 10,000 people, our employees are 20% white, 10% black, 10% Hispanic, 30% Asian, and 30% identify as other. We will take a sample of 1,000 employees and see if their race proportions are similar.

```
employee_races = (["white"]*2000) + (["black"]*1000) +\
                 (["hispanic"]*1000) + (["asian"]*3000) +\
                 (["other"]*3000)
```

- employee_races represents our employee population. For example, in our company of 10,000 people, 2,000 people are white (20%) and 3,000 people are Asian (30%).

- Let's take a random sample of 1,000 people, as shown:

```
demo_sample = random.sample(employee_races, 1000)
# Sample 1000 values
for race in set(demo_sample):
print( race + " proportion estimate:" )
print( demo_sample.count(race)/1000. )
```

- The output obtained would be as follows:

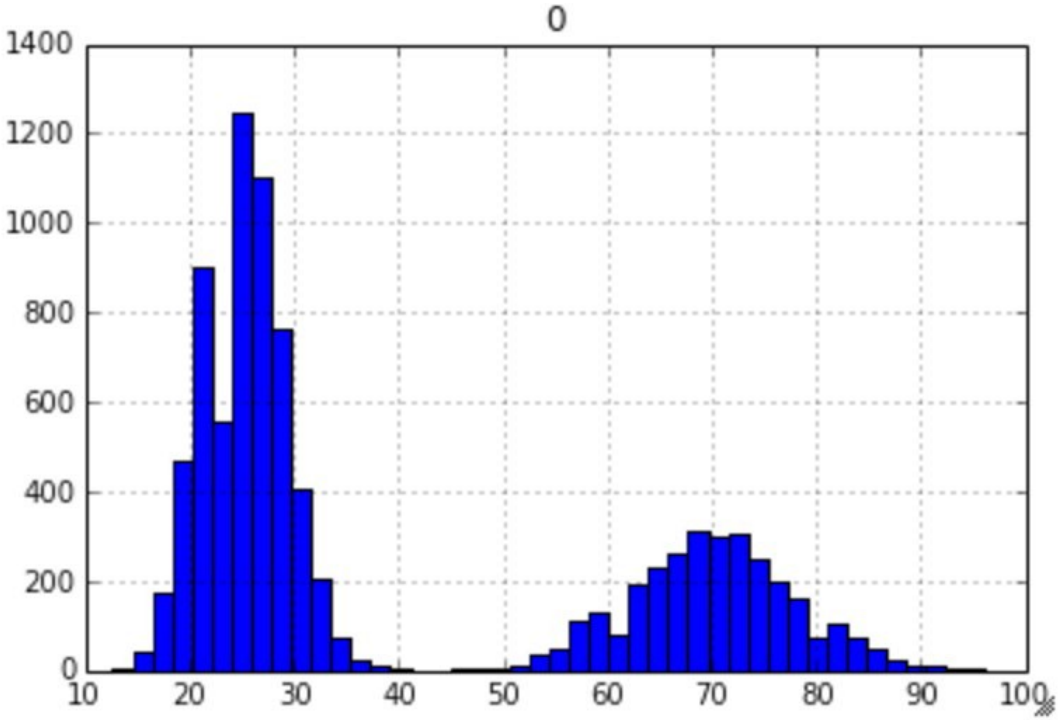
```
hispanic proportion estimate:
0.103
white proportion estimate:
0.192
other proportion estimate:
0.288
black proportion estimate:
0.1
asian proportion estimate:
0.317
```

- We can see that the race proportion estimates are very close to the underlying population's proportions. For example, we got 10.3% for Hispanic in our sample and the population proportion for Hispanic was 10%.

Sampling Distributions

- Many statistical tests rely on data that follows a normal pattern, and for the most part, a lot of real-world data is not normal (surprised?).
- Take our employee break data for example, you might think I was just being fancy creating data using the Poisson distribution, but I had a reason for this—I specifically wanted non-normal data, as shown:

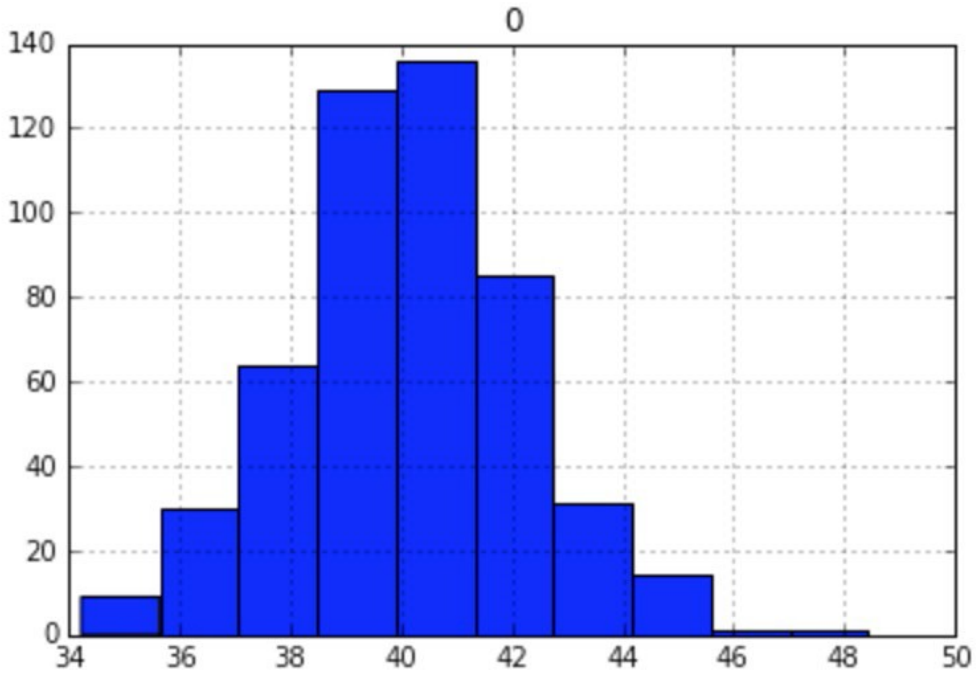
pd.DataFrame(breaks).hist(bins=50,range=(5,100))



- As you can see, our data is definitely not following a normal distribution, it appears to be **bi-modal**, which means that there are two peaks of break times, at around **25** and **70** minutes. As our data is not normal, many of the most popular statistics tests may not apply, however, if we follow the given procedure, we can create normal data!
- First off, we will need to utilize what is known as a **sampling distribution**, which is a distribution of point estimates of several samples of the same size. Our procedure for creating a sampling distribution will be the following:

  1. Take 500 different samples of the break times of size 100 each.
  2. Take a histogram of these 500 different point estimates (revealing their distribution).
- The number of elements in the sample (100) was arbitrary, but large enough to be a representative sample of the population. The number of samples I took (500) was also arbitrary, but large enough to ensure that our data would converge to a normal distribution:

```
point_estimates = []
for x in range(500): # Generate 500 samples
        sample = np.random.choice(a= breaks, size=100)
#take a sample of 100 points
point_estimates.append( sample.mean() )
# add the sample mean to our list of point estimates
pd.DataFrame(point_estimates).hist()
# look at the distribution of our sample means
```



- Behold! The sampling distribution of the sample mean appears to be normal even though we took data from an underlying bimodal population distribution.
- It is important to note that the bars in this histogram represent the average break length of 500 samples of employees, where each sample has 100 people in it. In other words, a sampling distribution is a distribution of several point estimates.
- Our data converged to a normal distribution because of something called the **central limit theorem**, which states that the sampling distribution (the distribution of point estimates) will approach a normal distribution as we increase the number of samples taken.
- What's more, as we take more and more samples, the mean of the sampling distribution will approach the true population mean, as shown:

```
breaks.mean() - np.array(point_estimates).mean()
# .047 minutes difference
```

- This is actually a very exciting result because it means that we can get even closer than a single point estimate by taking multiple point estimates and utilizing the central limit theorem!

Confidence Intervalso

- ✔For these reasons and more, we m
  TURNP0 of to a concept, known
  confidence intervals to find statistics.

3. A **confidence interval** is a range of values bas
   on a point estimate that contains the t
   population parameter at some confidence level

- **Confidence** is an important concept in advanced statistics. Its meaning is sometimes misconstrued.
- Informally, a confidence level does *not* represent a "probability of being correct"; instead, it represents the frequency that the obtained answer will be accurate. For example, if you want to have a 95% chance of capturing the true population parameter using only a single point estimate, we would have to set our confidence level to 95%.
- Calculating a confidence interval involves finding a point estimate, and then, incorporating a margin of error to create a range. The **margin of error** is a value that represents our certainty that our point estimate is accurate and is based on our desired confidence level, the variance of the data, and how big your sample is.
- There are many ways to calculate confidence intervals; for the purpose of brevity and simplicity, we will look at a single way of taking the confidence interval of a population mean. For this confidence interval, we need the following:
  - A point estimate. For this, we will take our sample mean of break lengths from our previous example.
  - An estimate of the population standard deviation, which represents the variance in the data.
    - This is calculated by taking the sample standard deviation (the standard deviation of the sample data) and dividing that number by the square root of the population size.
  - The degrees of freedom (which is the -1 sample size).

- Obtaining these numbers might seem arbitrary but, trust me, there is a reason for all of them. However, again for simplicity, I will use prebuilt Python modules, as shown, to calculate our confidence interval and, then, demonstrate its value:

```
sample_size = 100
# the size of the sample we wish to take
sample = np.random.choice(a= breaks, size = sample_size)
# a sample of sample_size taken from the 9,000 breaks population from before
sample_mean = sample.mean()
# the sample mean of the break lengths sample
sample_stdev = sample.std()
# sample standard deviation
sigma = sample_stdev/math.sqrt(sample_size)
# population standard deviation estimate
stats.t.interval(alpha = 0.95, # Confidence level 95%
df= sample_size - 1, # Degrees of freedom
loc = sample_mean, # Sample mean
scale = sigma) # Standard deviation
# (36.36, 45.44)
```

- To reiterate, this range of values (from 36.36 to 45.44) represents a confidence interval for the average break time with a 95% confidence.
- We already know that our population parameter is 39,99, and note that the interval includes the population mean of 39.99.
- I mentioned earlier that the confidence level was not a percentage of accuracy of our interval but the percent chance that the interval would even contain the population parameter at all.
- To better understand the confidence level, let's take 10,000 confidence intervals and see how often our population mean falls in the interval. First, let's make a function, as illustrated, that makes a single confidence interval from our breaks data:

```
# function to make confidence interval
def makeConfidenceInterval():
sample_size = 100
sample = np.random.choice(a= breaks, size = sample_size)
sample_mean = sample.mean()
# sample mean
sample_stdev = sample.std()
# sample standard deviation
sigma = sample_stdev/math.sqrt(sample_size)
# population Standard deviation estimate
return stats.t.interval(alpha = 0.95, df= sample_size - 1,
loc = sample_mean, scale = sigma)
```

- Now that we have a function that will create a single confidence interval, let's create a procedure that will test the probability that a single confidence interval will contain the true population parameter, 39.99:

  1. Take 10,000 confidence intervals of the sample mean.

  2. Count the number of times that the population parameter falls into our confidence intervals.

  3. Output the ratio of the number of times the parameter fell into the interval by 10,000:

```
times_in_interval = 0.
for i in range(10000):
interval = makeConfidenceInterval()
if 39.99 >= interval[0] and 39.99 <= interval[1]:
# if 39.99 falls in the interval
times_in_interval += 1
print times_in_interval / 10000
# 0.9455
```

- Success! We see that about 95% of our confidence intervals contained our actual population mean. Estimating population parameters through point estimates and confidence intervals is a relatively simple and powerful form of statistical inference.

- Let's also take a quick look at how the size of confidence intervals changes as we change our confidence level. Let's calculate confidence intervals for multiple confidence levels and look at how large the intervals are by looking at the difference between the two numbers. Our hypothesis will be that as we make our confidence level larger, we will likely see larger confidence intervals to be surer that we catch the true population parameter:

```
for confidence in (.5, .8, .85, .9, .95, .99):
confidence_interval = stats.t.interval(alpha = confidence, df=sample_size – 1, loc = sample_mean, scale = sigma)
length_of_interval = round(confidence_interval[1] - confidence_interval[0], 2)
# the length of the confidence interval
print "confidence {0} has a interval of size {1}".
format(confidence, length_of_interval)
```

confidence 0.5 has an interval of size 2.56
confidence 0.8 has an interval of size 4.88
confidence 0.85 has an interval of size 5.49
confidence 0.9 has an interval of size 6.29
confidence 0.95 has an interval of size 7.51
confidence 0.99 has an interval of size 9.94

- We can see that as we wish to be "more confident" in our interval, our interval expands in order to compensate.

Hypothesis Tests

- Hypothesis tests are one of the most widely used tests in statistics. They come in many forms; however, all of them have the same basic purpose.
- A **hypothesis test** is a statistical test that is used to ascertain whether we are allowed to assume that a certain condition is true for the entire population, given a data sample. Basically, a hypothesis test is a test for a certain hypothesis that we have about an entire population. The result of the test then tells us whether we should believe the hypothesis or reject it for an alternative one.
- You can think of the hypothesis tests' framework to determine whether the observed sample data deviates from what was to be expected from the population itself. Now this sounds like a difficult task but, luckily, Python comes to the rescue and includes built-in libraries to conduct these tests easily.

- A hypothesis test generally looks at two opposing hypotheses about a population. We call them the **null hypothesis** and the **alternative hypothesis**. The null hypothesis is the statement being tested and is the *default correct* answer; it is our starting point and our original hypothesis. The alternative hypothesis is the statement that opposes the null hypothesis. Our test will tell us which hypothesis we should trust and which we should reject.
- Based on sample data from a population, a hypothesis test determines whether or not to *reject* the null hypothesis. We usually use a **p-value** (which is based on our significance level) to make this conclusion.
- The following are some examples of questions you can answer with a hypothesis test:
  - Does the mean break time of employees differ from 40 minutes?
  - Is there a difference between people who interacted with website A and people who interacted with website B (A/B testing)?
  - Does a sample of coffee beans vary significantly in taste from the entire population of beans?

Conducting a Hypothesis Test

- There are multiple types of hypothesis tests out there, and among them are dozens of different procedures and metrics. Nonetheless, there are five basic steps that most hypothesis tests follow, which are as follows:

1. Specify the hypotheses:
   - Here, we formulate our two hypotheses: the null and the alternative
   - We usually use the notation of H_0 to represent the null hypothesis and H_a to represent our alternative hypothesis
2. Determine the sample size for the test sample:
   - This calculation depends on the chosen test. Usually, we have to determine a proper sample size in order to utilize theorems, such as the central limit theorem, and assume the normality of data.
3. Choose a significance level (usually called alpha or α):
   - A significance level of 0.05 is common

4. Collect the data:

- They collect a sample of data to conduct the test

5. Decide whether to reject or fail to reject the null hypothesis:

- This step changes slightly based on the type of test being used. The final result will either yield in rejecting the null hypothesis in favor of the alternative or failing to reject the null hypothesis.

In this chapter, we will look at the following three types of hypothesis tests:

- One-sample t-tests
- Chi-square goodness of fit
- Chi-square test for association/independence

- There are many more tests. However, these three are a great combination of distinct, simple, and powerful tests.
- One of the biggest things to consider when choosing which test we should implement is the type of data we are working with, specifically, are we dealing with continuous or categorical data. In order to truly see the effects of a hypothesis, I suggest we dive right into an example.
- First, let's look at the use of a t-tests to deal with continuous data.

One Sample t-tests

- The **one sample t-test** is a statistical test used to determine whether a quantitative (numerical) data sample differs *significantly* from another dataset (the population or another sample). Suppose, in our previous employee break time example, we look, specifically, at the engineering department's break times, as shown:

```
long_breaks_in_engineering = stats.poisson.rvs(loc=10, mu=55,
size=100)
short_breaks_in_engineering = stats.poisson.rvs(loc=10, mu=15,
size=300)
engineering_breaks = np.concatenate((long_breaks_in_engineering,
short_breaks_in_engineering))
print breaks.mean()
# 39.99
print engineering_breaks.mean()
# 34.825
```

- Note that I took the same approach as making the original break times, but with the following two differences:
  - I took a smaller sample from the Poisson distribution (to simulate that we took a sample of 400 people from the engineering department)
  - Instead of using a *mu* of 60 as before, I used 55 to simulate the fact that the engineering department's break behavior isn't exactly the same as the company's behavior as a whole
- It is easy to see that there seems to be a difference (of over 5 minutes) between the engineering department and the company as a whole.
- We usually don't have the entire population and the population parameters at our disposal, but I have them simulated in order to see the example work.
- So, even though we (the omniscient readers) can see a difference, we will assume that we know nothing of these population parameters and, instead, rely on a statistical test in order to ascertain these differences.

Example of a one sample t-tests

- Our objective here is to ascertain *whether there is a difference between the overall population's (company employees) break times and break times of employees in the engineering department*.
- Let us now conduct a t-test at a 95% confidence level in order to find a difference (or not!). Technically speaking, this test will tell us if the sample comes from the same distribution as the population.

Assumptions of the one sample t-tests

- Before diving into the five steps, we must first acknowledge that t-tests must satisfy the following two conditions to work properly:
  - The population distribution should be normal, or the sample should be *large* ($n \geq 30$).
  - In order to make the assumption that the sample is independently randomly sampled, it is sufficient to enforce that the population size should be at least 10 times larger than the sample size $(10n < N)$.
- Note that our test requires that either the underlying data be normal (which we know is not true for us), or that the sample size be at least 30 points large.
- For the t-test, this condition is sufficient to assume normality. This test also requires independence, which is satisfied by taking a sufficiently *small* sample. Sounds weird, right?
- The basic idea is that our sample must be large enough to assume normality (through conclusions similar to the central limit theorem) but small enough as to be *independent* from the population.

- Now, let's follow our five steps:

1. Specify the hypotheses.

   We will let H_0 = *the engineering department takes breaks the same as the company as a whole*

   If we let this be the company average, we may write:

   $$H_0:$$

   Note how this is our *null*, or *default*, hypothesis. It is what we would assume, given no data. What we would like to show is the **alternative hypothesis**.

   Now that we actually have some options for our alternative, we could either say that the engineering mean (let's call it that) is lower than the company average, higher than the company average, or just flat out different (higher or lower) than the company average:

   - If we wish to answer the question, *is the sample mean different from the company average*, then this is called a **two-tailed test** and our alternative hypothesis would be as follows:

     $$H_a:$$

- If we want to answer *either is the sample mean lower than the company average or is the sample mean higher than the company average*, then we are dealing with a **one-tailed test** and our alternative hypothesis would be one or the other of the following hypotheses:

  Ha:(engineering takes longer breaks)

  Ha:(engineering takes shorter breaks)

- The difference between one and two tails is the difference of dividing a number later on by 2 or not. The process remains completely unchanged for both. For this example, let's choose the two-tailed test. So, we are testing for whether or not this sample of the engineering department's average break times is different from the company average.

2. Determine the sample size for the test sample.

As mentioned earlier, most tests (including this one) make the assumption that either the underlying data is normal or that our sample is in the right range.

- The sample is at least 30 points (it is 400)
- The sample is less than 10% of the population (which would be 900 people)

3. Choose a significance level (usually called alpha or α).

We will choose a 95% significance level, which means that our alpha would actually be *1 - .95 = .05*

4. Collect the data.

Done! This was generated through the two Poisson distributions

5. Decide whether to reject or fail to reject the null hypothesis.

As mentioned before, this step varies based on the test used. For a one sample t-test, we must calculate two numbers: the test statistic and our p value. Luckily, we can do this in one line in Python.

- A test statistic is a value that is derived from sample data during a type of hypothesis test. They are used to determine whether or not to reject the null hypothesis.
- The test statistic is used to compare the observed data with what is expected under the null hypothesis. The test statistic is used in conjunction with the p-value.
- The p-value is the probability that the observed data occurred this way by chance.

- When the data is showing very strong evidence against the null hypothesis, the test statistic becomes large (either positive or negative) and the p-value usually becomes very small, which means that our test is showing powerful results and what is happening is, probably, not happening by chance.
- In the case of a t-test, a *t value* is our test statistic, as shown:

  t_statistic, p_value = stats.ttest_1samp(a= engineering_breaks,
  popmean= breaks.mean())
- We input the engineering_breaks variable (which holds 400 break times) and the population mean, and we obtain the following numbers:

  t_statistic == -5.742

  p_value == .00000018

- The test result shows that the *t value* is -5.742. This is a standardized metric that reveals the deviation of the sample mean from the null hypothesis.
- The p value is what gives us our final answer. Our p-value is telling us how often our result would appear by chance.
- So, for example, if our p-value was .06, then that would mean we should expect to observe this data by chance about 6% of the time.
- This means that about 6% of samples would yield results like this.
- We are interested in how our p-value compares to our significance level:
  - If the p-value is *less than* the significance level, then we can *reject* the null hypothesis
  - If the p-value is *greater than* the significance level, then we *failed to reject* the null hypothesis

- Our p value is way lower than .05 (our chosen significance level), which means that we may reject our null hypothesis in favor for the alternative. This means that the engineering department seems to take different break lengths than the company as a whole!
- There are many other types of t-tests, including one-tailed tests (mentioned before) and paired tests as well as two sample t-tests (both not mentioned yet).
- These procedures can be readily found in statistics literature; however, we should look at something very important—what happens when we get it wrong

Type I and Type II errors

- A type I error occurs if we reject the null hypothesis when it is actually true. This is also known as a **false positive**. The type I error rate is equal to the significance level α, which means that if we set a higher confidence level, for example, a significance level of 99%, our α is .01, and therefore our false positive rate is 1%.
- A type II error occurs if we fail to reject the null hypothesis when it is actually false. This is also known as a **false negative**. The higher we set our confidence level, the more likely we are to actually see a type II error.

Hypothesis test for categorical variables

- T-tests (among other tests) are hypothesis tests that work to compare and contrast quantitative variables and underlying population distributions. In this section, we will explore two new tests, both of which serve to explore qualitative data. They both are a form of test called chi-square tests. These two tests will perform the following two tasks for us:
  - Determine whether a sample of categorical variables is taken from a specific population (similar to the t-test)
  - Determine whether two variables affect each other and are associated to each other.

Chi-Square Goodness of fit test

- The one-sample t-test was used to check whether a sample mean differed from the population mean. The chi-square goodness of fit test is very similar to the one sample t-test in that it tests whether the distribution of the sample data matches an expected distribution, while the big difference is that it is testing for categorical variables.
- For example, a chi-square goodness of fit test would be used to see if the race demographics of your company match that of the entire city of the U.S. population. It can also be used to see if users of your website show similar characteristics to average Internet users.
- As we are working with categorical data, we have to be careful because categories like "male", "female," or "other" don't have any mathematical meaning. Therefore, we must consider counts of the variables rather than the actual variables themselves.
- In general, we use the chi-square goodness of fit test in the following cases:
  - We want to analyze one categorical variable from one population
  - We want to determine if a variable fits a specified or expected distribution
- In a chi-square test, we compare what is observed to what we expect.

Assumptions of the Chi-Square Goodness of fit test

- There are two usual assumptions of this test, as follows:
  - All the expected counts are at least 5
  - Individual observations are independent and the population should be at least 10 times as large as the sample, ($10n < N$)
- The second assumption should look familiar to the t-test; however, the first assumption should look foreign. Expected counts are something we haven't talked about yet but are about to!

- When formulating our null and alternative hypotheses for this test, we consider a default distribution of categorical variables. For example, if we have a die and we are testing whether or not the outcomes are coming from a fair die, our hypothesis might look as follows:

H_0: The specified distribution of the categorical variable is correct.

p1 = 1/6, p2 = 1/6, p3 = 1/6, p4 = 1/6, p5 = 1/6, p6 = 1/6

- Our alternative hypothesis is quite simple, as shown:
- H_a : The specified distribution of the categorical variable is not correct. At least one of the *pi* values is not correct.
- In the t-test, we used our test statistic (the t value) to find our p-value. In a chi-square test, our test statistic is, well, chi-square.

  Test Statistic: $\chi 2$ = over k categories

  Degrees of Freedom = k − 1
- A critical value is when we use $\chi 2$ as well as our degrees of freedom and our significance level, and then reject the null hypothesis if the p-value is below our significance level (the same as before).
- Let's see an example to understand further.

Example of a Chi-Square test for Goodness of fit

- The CDC categorizes adult BMIs into four classes: Under/Normal, Over weight, Obesity, and Extreme Obesity. A 2009 survey showed the distribution for adults in the U.S. to be 31.2%, 33.1%, 29.4%, and 6.3% respectively. A total of 500 adults are randomly sampled and their BMI categories are recorded. Is there evidence to suggest that BMI trends have changed since 2009? Test at the 0.05 significance level.

	Under/Normal	Over	Obesity	Extreme Obesity	Total
Observed	102	178	186	34	500

- First, let's calculate our expected values. In a sample of 500, we *expect* 156 to be Under/Normal (that's 31.2% of 500), and we fill in the remaining boxes in the same way.

	Under/Normal	Over	Obesity	Extreme Obesity	Total
Observed	102	178	186	34	500
Expected	156	165.5	147	31.5	500

- First, check the conditions:
  - All of the expected counts are greater than 5
  - Each observation is independent and our population is very large (much more than 10 times of 500 people)
- Next, carry out a goodness of fit test. We will set our null and alternative hypotheses:
  - *H***0** : The 2009 BMI distribution is still correct.
  - *Ha* : The 2009 BMI distribution is no longer correct (at least one of the proportions is different now). We can calculate our test statistic by hand:

$$\text{Test Statistic: } \chi^2 = \sum \frac{(Observed - Expected)^2}{Expected} \text{ for } df = 3$$

$$= \frac{(102-156)^2}{156} + \frac{(178-165.5)^2}{165.5} + \frac{(186-147)^2}{147} + \frac{(34-31.5)^2}{31.5} = 30.18$$

- Alternatively, we can use our handy dandy Python skills, as shown:

  observed = [102, 178, 186, 34]

  expected = [156, 165.5, 147, 31.5]

  chi_squared, p_value = stats.chisquare(f_obs= observed, f_exp= expected)

  chi_squared, p_value

  #(30.1817679275599, 1.2637431031106e-06)

- Our p-value is lower than .05; therefore, we may reject the null hypothesis in favor of the fact that the BMI trends today are different from what they were in 2009.

Chi-Square test for Association/Independence

- Independence as a concept in probability is when knowing the value of one variable tells you nothing about the value of another. For example, we might expect that the country and the month you were born in are independent.
- However, knowing which type of phone you use might indicate your creativity levels. Those variables might not be independent.
- The chi-square test for association/independence helps us ascertain whether two categorical variables are independent of one another. The test for independence is commonly used to determine whether variables like education levels or tax brackets vary based on demographic factors, such as gender, race, and religion.
- Let's look back at an example posed in the preceding chapter, the A/B split test.

- Recall that we ran a test and exposed half of our users to a certain landing page (Website A), exposed the other half to a different landing page (Website B), and then, measured the sign up rates for both sites. We obtained the following results:

	Did not sign up	**Signed up**
Website A	134	54
Website B	110	48

Results of our A/B test

- We calculated website conversions but what we really want to know is whether there is a difference between the two variables: *which website was the user exposed to?* and *did the user sign up?*. For this, we will use our chi-square test.

Assumptions of the Chi-Square Independence test

- There are the following two assumptions of this test:
  - All expected counts are at least 5
  - Individual observations are independent and the population should be at least 10 times as large as the sample, ($10n < N$)
- Note that they are exactly the same as the last chi-square test.
- Let's set up our hypotheses:
  - H_0 : There is no association between two categorical variables in the population of interest
  - H_0 : Two categorical variables are independent in the population of interest
  - H_a : There is an association between two categorical variables in the population of interest
  - H_a : Two categorical variables are not independent in the population of interest

- You might notice that we are missing something important here. Where are the expected counts? Earlier, we had a prior distribution to compare our observed results to but now we do not. For this reason, we will have to create some. We can use the following formula to calculate the expected values for each value. In each cell of the table, we can use:

Expected Count = *to calculate our chi-square test statistic and our degrees of freedom*

$$\text{Test Statistic: } \chi^2 = \sum \frac{(Observed_{r,c} - Expected_{r,c})^2}{Expected_{r,c}}$$

over r rows and c columns

$$\text{Degrees of Freedom} = (r-1) \cdot (c-1)$$

- Here, *r* is the number of rows and *c* is the number of columns. Of course, as before, when we calculate our p-value, we will reject the null if that p-value is less than the significance level. Let's use some built-in Python methods, as shown, in order to quickly get our results:

```
observed = np.array([[134, 54],[110, 48]])
# built a 2x2 matrix as seen in the table above
chi_squared, p_value, degrees_of_freedom, matrix = stats.chi2_contingency(observed= observed)
chi_squared, p_value
# (0.047626923694910 45, 0.8272452870442226 2)
```

- We can see that our p-value is quite large; therefore, we *fail* to reject our null hypothesis and we cannot say for sure that seeing a particular website has any effect on a user's sign up. There is no association between these variables.